

조도센서를 활용한 스마트 커튼 개폐 장치

20225250 스마트 IoT 최우진

목차

01 배경 및 아이디어

02 프로젝트 내용

03 하드웨어 및
소프트웨어 구현

04 Demo 영상

05 레퍼런스

배경 및 제안 아이디어



현대인들은 불규칙한 수면 패턴과 과도한 인공 조명 사용으로 인해 수면의 질 저하 및 에너지 낭비 문제를 겪고 있음. 기존 스마트홈 커튼 시스템은 다음과 같은 한계점이 존재함을 확인함.

배경 및 제안 아이디어

기존 방법의 문제점

높은 비용 : 상용 스마트 커튼 시스템은 설치 비용이 수십만 원에 달해 접근성이 낮음.

수동 제어 의존 : 대부분의 전동 커튼은 리모컨이나 앱으로 수동 제어해야 하며, 자동화 기능이 제한적임.

조명 에너지 낭비 : 낮 시간에도 커튼이 닫혀있어 불필요하게 전등을 사용하는 경우가 많음.

제안 아이디어

본 프로젝트는 조도센서를 활용하여 주변 환경의 밝기를 자동으로 감지하고, 서보모터로 커튼을 자동으로 개폐함으로써 자연 채광을 최대한 활용하고, 쾌적한 생활 환경을 제공함.

배경 및 제안 아이디어

기대 효과

자동화 된 커튼 제어 : 조도센서로 측정한 밝기에 따라 커튼을 자동으로 개폐하여 사용자 편의성 증대

에너지 절약 : 낮 시간 자연 채광 활용으로 전등 사용 감소 및 전기료 절감

CoAP 기반 IoT 통신 : CoAP 프로토콜의 GET / PUT 메시지 및 Observe 옵션을 활용한 실시간 센서 모니터링 구현

원격 제어 기능 : UI를 통한 수동 제어 및 자동제어 전환 가능

프로젝트 내용

- 시스템 구조 -

- 라즈베리 파이 : CoAP 서버 역할 수행 및 센서, 액추에이터 제어
- 조도센서(GY-30) : 주변 밝기 감지, GPIO에서 아날로그 값 읽기
- 서보모터(SG90) : 커튼 개폐 동작 수행, PWM 신호로 각도 제어
- LCD 디스플레이 : 현재 개폐 상태를 표시와 오토 모드인지 수동인지 표시함.
- CoAP 서버 : CoAP 통신을 이용한 Resource 관리

하드웨어 및 소프트웨어 구현

하드웨어 구현

서보모터 (SG90)

VCC (빨간선): 5V (브레드보드 +라인)

GND (갈색선): Ground (브레드보드 -라인)

Signal (주황선): 8번핀

LCD 디스플레이 (I2C 16×2)

VCC: 5V (브레드보드 +라인)

GND: Ground (브레드보드 -라인)

SDA: I2C SDA (3번 핀) - 조도센서와 공유

SCL: I2C SCL (5번 핀) - 조도센서와 공유

2번핀 브레드보드 +라인

3번핀 SDA

5번핀 SCL

6번핀 브레드보드 -라인

조도센서 (GY-30)

VCC: 3.3V (1번 핀)

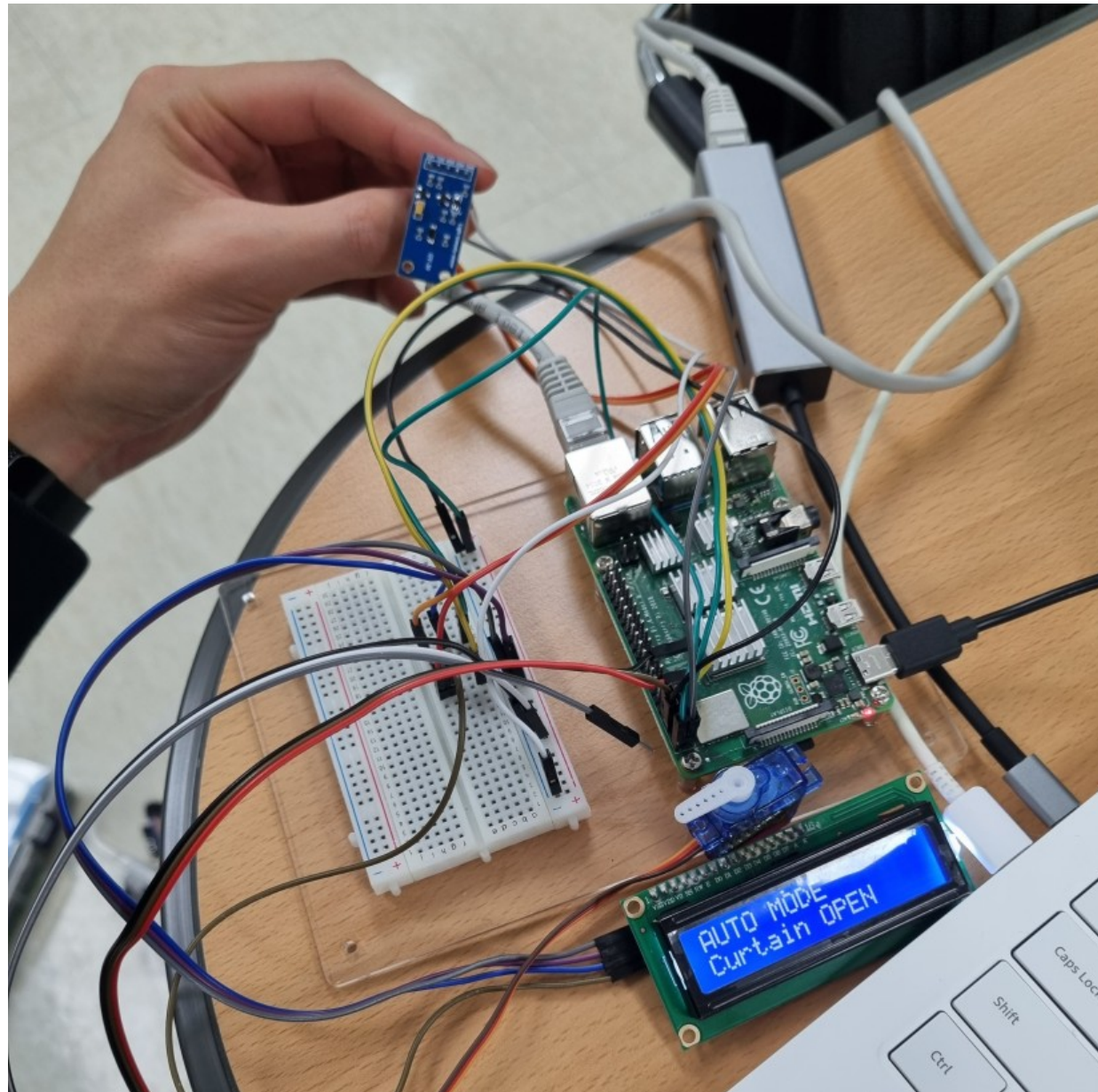
GND: Ground (브레드보드 -라인)

SDA: I2C SDA (3번 핀)

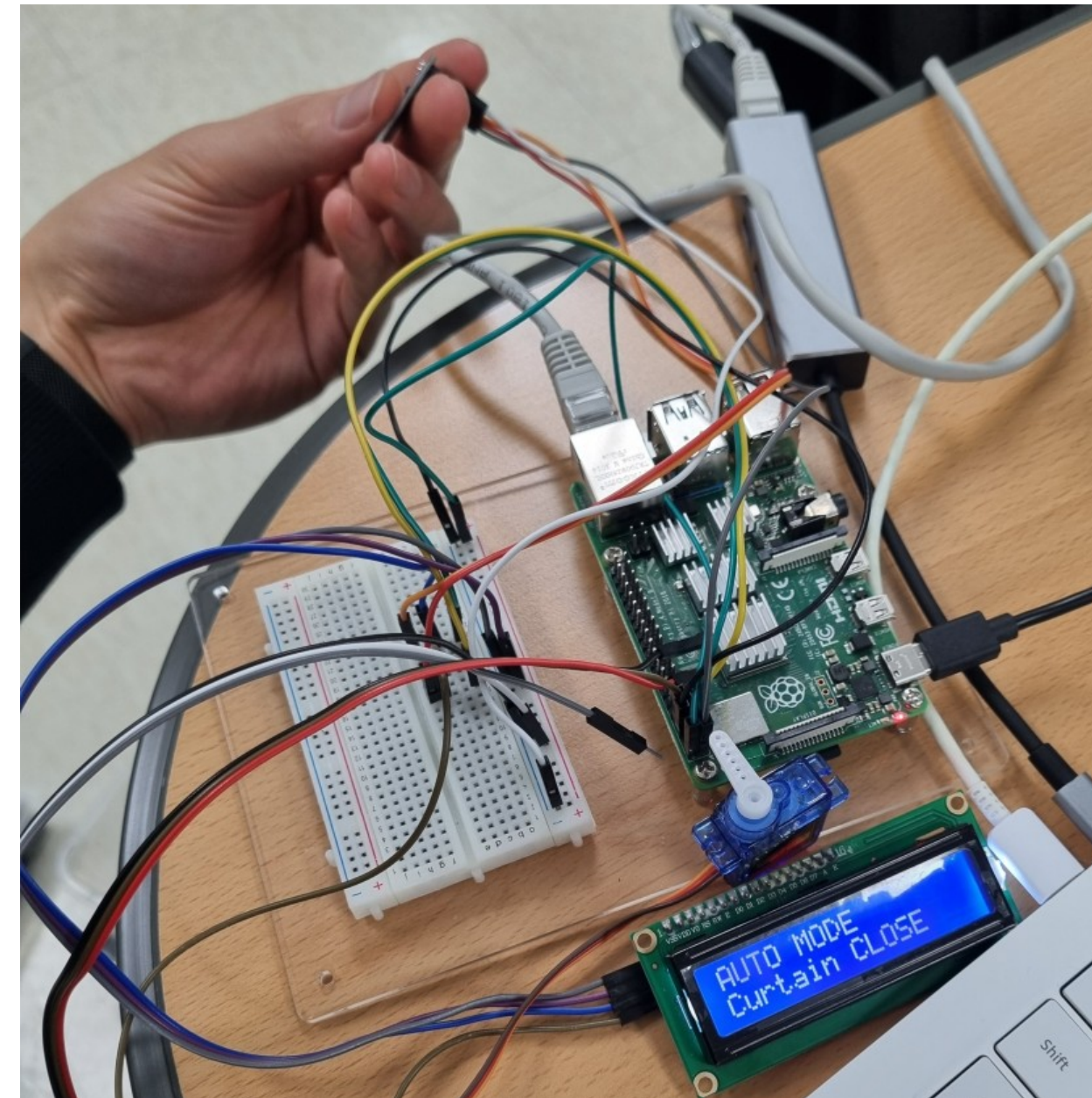
SCL: I2C SCL (5번 핀)

하드웨어 및 소프트웨어 구현

하드웨어 구현



밝을때 구현 모습



어두울 때 구현모습

하드웨어 및 소프트웨어 구현

소프트웨어 구현



소프트웨어 구현

- Light_sensor.java

- 클래스 변수로 I2C 통신으로 조도 센서값을 읽어올 수 있는 객체 선언
- GET 메시지 수신시, GY-30 센서를 통해 조도를 감지하고 결과를 Payload에 표시하여 응답
- optional_changed() 메소드에서 조도값에 따라 자동으로 커튼 개폐 제어

```
public class Light_sensor extends BasicCoapResource {  
  
    private float lightLevel = 0.0f;  
    private I2CDevice GY30;  
    private static final int DEVICE_ADDRESS = 0x23;  
    private static final byte CONTINUOUS_HIGH_RES_MODE = 0x10;  
  
    private static final float LIGHT_THRESHOLD = 300.0f;  
    private String lastAction = "none";  
    private Servo_motor servoMotor;  
    private LCD_Display lcd;  
  
    private Light_sensor(String path, String value, CoapMediaType mediaType) {  
        super(path, value, mediaType);  
    }  
  
    public Light_sensor() {  
        this("/light", "0.0", CoapMediaType.text_plain);  
        try {  
            I2CBus bus = I2CFactory.getInstance(I2CBus.BUS_1);  
            GY30 = bus.getDevice(DEVICE_ADDRESS);  
            System.out.println("✓ Light sensor initialized");  
        } catch (Exception e) {  
            System.err.println("✗ Light sensor initialization error: " + e.getMessage());  
            e.printStackTrace();  
        }  
    }  
  
    @Override  
    public synchronized CoapData get(List mediaTypesAccepted) {  
        this.lightLevel = readLightLevel();  
        String value = String.valueOf(this.lightLevel);  
        return new CoapData(Encoder.StringToByte(value), CoapMediaType.text_plain);  
    }  
}
```

Get 메시지 수신 시 동작 정의

클래스 변수로 I2C 장치선언 및 생성자에서 조도센서 초기화

```

public void optional_changed() {
    this.lightLevel = readLightLevel();
    if (this.lightLevel < 0) { return; }

    System.out.println("Light level: " + this.lightLevel + " lux");

    // 자동 제어 로직 (반대로 변경)
    if (servoMotor != null) {
        // 밝을 때 (아침) → 커튼 열기
        if (this.lightLevel >= LIGHT_THRESHOLD && !lastAction.equals("open")) {
            System.out.println("_____");
            System.out.println("☉ AUTO MODE: Bright detected!");
            System.out.println("  Light: " + this.lightLevel + " lux >= " + LIGHT_THRESHOLD + " lux");
            System.out.println("  → Opening curtain...");

            byte[] openCommand = Encoder.StringToByte("open");
            servoMotor.setValue(openCommand);

            if (lcd != null) {
                byte[] lcdText = Encoder.StringToByte("AUTO MODE|Curtain OPEN");
                lcd.setValue(lcdText);
            }

            lastAction = "open";
            System.out.println("✓ Auto control completed");
            System.out.println("_____");
        }
    }
}

```

밝을때의 커튼 제어

어두울때의 커튼 제어

```

// 어두울 때 (밤) → 커튼 닫기
} else if (this.lightLevel < LIGHT_THRESHOLD && !lastAction.equals("close")) {
    System.out.println("_____");
    System.out.println("🌙 AUTO MODE: Dark detected!");
    System.out.println("  Light: " + this.lightLevel + " lux < " + LIGHT_THRESHOLD + " lux");
    System.out.println("  → Closing curtain...");

    byte[] closeCommand = Encoder.StringToByte("close");
    servoMotor.setValue(closeCommand);

    if (lcd != null) {
        byte[] lcdText = Encoder.StringToByte("AUTO MODE|Curtain CLOSE");
        lcd.setValue(lcdText);
    }

    lastAction = "close";
    System.out.println("✓ Auto control completed");
    System.out.println("_____");
}
}

this.changed();
}

```

소프트웨어 구현

- LCD_Display.java

- 생성자에서 LCD 모듈 사용을 위한 I2C통신 초기화
- PUT 메시지 수신시, 수신한 메시지의 Payload를 LCD에 표시
- "|" 문자로 첫 번째 줄과 두 번째 줄 구분하여 2줄 출력

```
public LCD_Display() {
    this("/lcd", "", CoapMediaType.text_plain);
    try {
        System.out.println("Initializing LCD Display...");

        // I2C 버스 초기화
        I2CBus bus = I2CFactory.getInstance(I2CBus.BUS_1);
        lcd = bus.getDevice(LCD_ADDRESS);

        // LCD 초기화
        Thread.sleep(50);
        lcdByte(0x33, LCD_CMD);
        lcdByte(0x32, LCD_CMD);
        lcdByte(0x06, LCD_CMD);
        lcdByte(0x0C, LCD_CMD);
        lcdByte(0x28, LCD_CMD);
        lcdByte(0x01, LCD_CMD);
        Thread.sleep(5);

        System.out.println("✓ LCD initialized at address 0x" + Integer.toHexString(LCD_ADDRESS));
    } catch (Exception e) {
        System.err.println("✗ LCD initialization error: " + e.getMessage());
        e.printStackTrace();
    }
}
```

LCD모듈 초기화

```

@Override
public synchronized boolean setValue(byte[] value) {
    this.text = Encoder.ByteToString(value);

    System.out.println("=====");
    System.out.println("LCD setValue() called");
    System.out.println("Received text: [" + this.text + "]");

    try {
        if (lcd == null) {
            System.err.println("X ERROR: LCD not initialized!");
            return false;
        }

        // LCD 화면 지우기
        System.out.println("→ Clearing LCD...");
        lcdByte(0x01, LCD_CMD);
        Thread.sleep(5);

        // "|"로 줄 구분
        String[] lines = this.text.split("\\|");
        System.out.println("  Split into " + lines.length + " line(s)");

        // 첫 번째 줄
        if (lines.length > 0) {
            System.out.println("→ Writing line 1: [" + lines[0] + "]");
            lcdByte(0x80, LCD_CMD); // 첫 번째 줄
            lcdString(lines[0]);
            System.out.println("  ✓ Line 1 written");
        }
    }

```

```

        // 두 번째 줄
        if (lines.length > 1) {
            System.out.println("→ Writing line 2: [" + lines[1] + "]");
            lcdByte(0xC0, LCD_CMD); // 두 번째 줄
            lcdString(lines[1]);
            System.out.println("  ✓ Line 2 written");
        }

        System.out.println("✓ LCD updated successfully");
    } catch (Exception e) {
        System.err.println("X LCD write error: " + e.getMessage());
        e.printStackTrace();
    }

    System.out.println("=====");

    return true;
}

```

PUT 메시지 수신 시 동작 정의

소프트웨어 구현

- Servo_motor.java

- 클래스 변수로 PWM 출력을 제어할 수 있는 pi4j 객체 선언
- PUT 메시지 수신 시, "open" 또는 "close" 명령에 따라 서보모터 각도 제어
- setServoAngle() 메소드로 PWM 신호를 이용한 각도 제어 (0도/90도)

```
public class Servo_motor extends BasicCoapResource {

    private String state = "closed";
    private boolean initialized = false;
    private GpioController gpio;
    private GpioPinPwmOutput pwm;

    private Servo_motor(String path, String value, CoapMediaType mediaType) {
        super(path, value, mediaType);
    }

    public Servo_motor() {
        this("/servo", "closed", CoapMediaType.TEXT_PLAIN);
        try {
            System.out.println("Initializing Servo Motor...");

            // GPIO 초기화
            gpio = GpioFactory.getInstance();

            // 물리적 핀 8번 = BCM GPIO 14 = WiringPi 15번
            pwm = gpio.provisionSoftPwmOutputPin(RaspiPin.GPIO_15);

            // PWM 범위 설정
            pwm.setPwmRange(200);

            // 초기 위치
            setServoAngle(0);

            initialized = true;
            System.out.println("✓ Servo motor initialized");
        } catch (Exception e) {
            System.err.println("X Servo initialization error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

서보모터 초기화

소프트웨어 구현

```
private void setServoAngle(int angle) {
    try {
        // PWM Range 200 기준 (100us * 200 = 20ms)
        // 0도: 0.5ms = 5
        // 90도: 1.5ms = 15
        // 180도: 2.5ms = 25

        int pwmValue;
        if (angle == 0) {
            pwmValue = 5;    // 0도 - CLOSE (0.5ms)
        } else if (angle == 90) {
            pwmValue = 15;   // 90도 - OPEN (1.5ms)
        } else {
            // 다른 각도 계산
            pwmValue = 5 + (angle * 20 / 180);
        }

        System.out.println(" Angle: " + angle + "° → PWM: " + pwmValue);
        pwm.setPwm(pwmValue);

        // 서보가 목표 위치에 도달할 시간
        Thread.sleep(1000);

        System.out.println(" Done");
    } catch (Exception e) {
        System.err.println("X Error: " + e.getMessage());
        e.printStackTrace();
    }
}
```

```
@Override
public synchronized boolean setValue(byte[] value) {
    this.state = Encoder.ByteToString(value).trim().toLowerCase();
    System.out.println("_____");
    System.out.println("Servo: [" + this.state + "]");

    if (!initialized) {
        System.err.println("X Not initialized!");
        return false;
    }

    if (this.state.equals("open")) {
        System.out.println("→ OPEN (90°)");
        setServoAngle(90);
    } else if (this.state.equals("close")) {
        System.out.println("→ CLOSE (0°)");
        setServoAngle(0);
    } else {
        System.err.println("X Unknown: [" + this.state + "]");
        System.out.println("_____");
        return false;
    }

    System.out.println("_____");
    return true;
}
```

서보모터 각도 전환 부분

소프트웨어 구현

- Smart_Curtain_Server.java

- 서버에 조도센서, 서보모터, LCD 리소스 추가
- 조도센서에 Observe 옵션 활성화하여 실시간 모니터링 구현
- 조도센서와 서보모터, LCD를 연결하여 자동 제어 로직 구현
- 1초마다 조도 값을 체크하며 자동으로 커튼 개폐 동작 수행

```
public class Smart_Curtain_Server {  
    private static Smart_Curtain_Server coapServer;  
    private CoapResourceServer resourceServer;  
  
    public static void main(String[] args) {  
        coapServer = new Smart_Curtain_Server();  
        coapServer.start();    }  
}
```

리소스 초기화

소프트웨어 구현

```
public void start() {
    System.out.println("=====");
    System.out.println("    Smart Curtain System Server");
    System.out.println("    AUTO MODE Enabled");
    System.out.println("=====");

    if (this.resourceServer != null)
        this.resourceServer.stop();
    this.resourceServer = new CoapResourceServer();

    Light_sensor light_sensor = new Light_sensor();
    Servo_motor servo = new Servo_motor();
    LCD_Display lcd = new LCD_Display();

    light_sensor.setServoMotor(servo);
    light_sensor.setLCD(lcd);

    light_sensor.registerServerListener(resourceServer);

    this.resourceServer.createResource(light_sensor);
    this.resourceServer.createResource(servo);
    this.resourceServer.createResource(lcd);

    try {
    } catch (Exception e) {
    }

    while (true) {
        try {
            Thread.sleep(1000);
            light_sensor.optional_changed();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

조도센서와 서보모터 구현 부분

소프트웨어 구현 - GUI_client.java

- Java Swing을 이용한 GUI 기반 CoAP 클라이언트 구현
- 서버 연결 시 CoapChannelManager로 라즈베리 파이 서버와 통신 채널 생성
- Observe 기능으로 조도센서 실시간 모니터링 (Token: "lightToken")
- 수동/자동 모드 전환 및 커튼 제어 기능 제공

```
public GUI_client(String serverAddress, int serverPort) {
    super("Smart Curtain System");
    this.setLayout(null);

    String sAddress = serverAddress;
    int sPort = serverPort;

    CoapChannelManager channelManager = BasicCoapChannelManager.getInstance();
    try {
        clientChannel = channelManager.connect(this, InetAddress.getByAddress(sAddress), sPort);
    } catch (UnknownHostException e) {
        e.printStackTrace();
        System.exit(-1);
    }

    if (null == clientChannel) {
        return;
    }

    // UI 레이아웃 설정
    setupUI();

    // 창 크기 및 표시
    this.setSize(800, 700);
    this.setVisible(true);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
```

생성자 서버에서 연결 및 UI구성

소프트웨어 구현

```
private void setupUI() {  
    // 상태 표시 레이블  
    light_label.setBounds(20, 20, 300, 30);  
    light_label.setFont(new Font("Arial", Font.BOLD, 16));  
  
    curtain_label.setBounds(20, 60, 300, 30);  
    curtain_label.setFont(new Font("Arial", Font.BOLD, 16));  
  
    mode_label.setBounds(20, 100, 300, 30);  
    mode_label.setFont(new Font("Arial", Font.BOLD, 16));  
  
    // 버튼  
    btn_observe.setBounds(20, 150, 180, 40);  
    btn_auto.setBounds(220, 150, 180, 40);  
    btn_manual.setBounds(420, 150, 180, 40);  
  
    btn_open.setBounds(20, 210, 180, 40);  
    btn_close.setBounds(220, 210, 180, 40);  
  
    // 로그 디스플레이  
    display_label.setBounds(20, 270, 200, 20);  
    display_text.setLineWrap(true);  
    display_text.setFont(new Font("Arial", Font.PLAIN, 12));  
    display_text_jp.setBounds(20, 300, 740, 330);  
}
```

UI 컴포넌트의 구성

소프트웨어 구현

```
// 버튼 이벤트 리스너
btn_observe.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (!observing) {
            CoapRequest request = clientChannel.createRequest(CoapRequestCode.GET, "/light", true);
            request.setToken(Encoder.StringToByte("lightToken"));
            request.setObserveOption(0);
            displayRequest(request);
            clientChannel.sendMessage(request);
            observing = true;
            btn_observe.setText("OBSERVING...");
            display_text.append("Started observing light sensor\n");
        }
    }
});

btn_auto.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        autoMode = true;
        mode_label.setText("Mode: AUTO");
        display_text.append("Switched to AUTO mode\n");
    }
});

btn_manual.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        autoMode = false;
        mode_label.setText("Mode: MANUAL");
        display_text.append("Switched to MANUAL mode\n");
    }
});
```

오토와 매뉴얼 모드

소프트웨어 구현

```
btn_open.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        CoapRequest request = clientChannel.createRequest(CoapRequestCode.PUT, "/servo", true);
        request.setPayload(new CoapData("open", CoapMediaType.text_plain));
        displayRequest(request);
        clientChannel.sendMessage(request);

        // LCD 업데이트
        updateLCD("Curtain: OPEN");
    }
});

btn_close.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        CoapRequest request = clientChannel.createRequest(CoapRequestCode.PUT, "/servo", true);
        request.setPayload(new CoapData("close", CoapMediaType.text_plain));
        displayRequest(request);
        clientChannel.sendMessage(request);

        // LCD 업데이트
        updateLCD("Curtain: CLOSE");
    }
});
```

소프트웨어 구현

```
@Override
public void onResponse(CoapClientChannel channel, CoapResponse response) {
    if (response.getPayload() != null) {
        display_text.append("Response: " + response.toString() + " payload: "
            + Encoder.ByteToString(response.getPayload()) + "\n");
        display_text.setCaretPosition(display_text.getDocument().getLength());
    } else {
        display_text.append("Response: " + response.toString() + "\n");
        display_text.setCaretPosition(display_text.getDocument().getLength());
    }

    // Observe 토큰 확인 - 조도 센서 데이터
    if (Encoder.ByteToString(response.getToken()).equals("lightToken")) {
        float lux = Float.parseFloat(Encoder.ByteToString(response.getPayload()));
        light_label.setText("Light Level: " + String.format("%.1f", lux) + " lux");

        // 자동 모드일 때 커튼 제어
        if (autoMode) {
            control_curtain(lux);
        }
    }
}
```

CoAP 응답 수신 시 처리

소프트웨어 구현

```
// 조도에 따른 자동 커튼 제어
public void control_curtain(float lux) {
    CoapRequest request = clientChannel.createRequest(CoapRequestCode.PUT, "/servo", true);

    if (lux > 700.0) {
        // 밝음 - 커튼 열기
        request.setPayload(new CoapData("open", CoapMediaType.text_plain));
        curtain_label.setText("Curtain: OPEN");
        updateLCD("Light: " + String.format("%.0f", lux) + "|Curtain: OPEN");

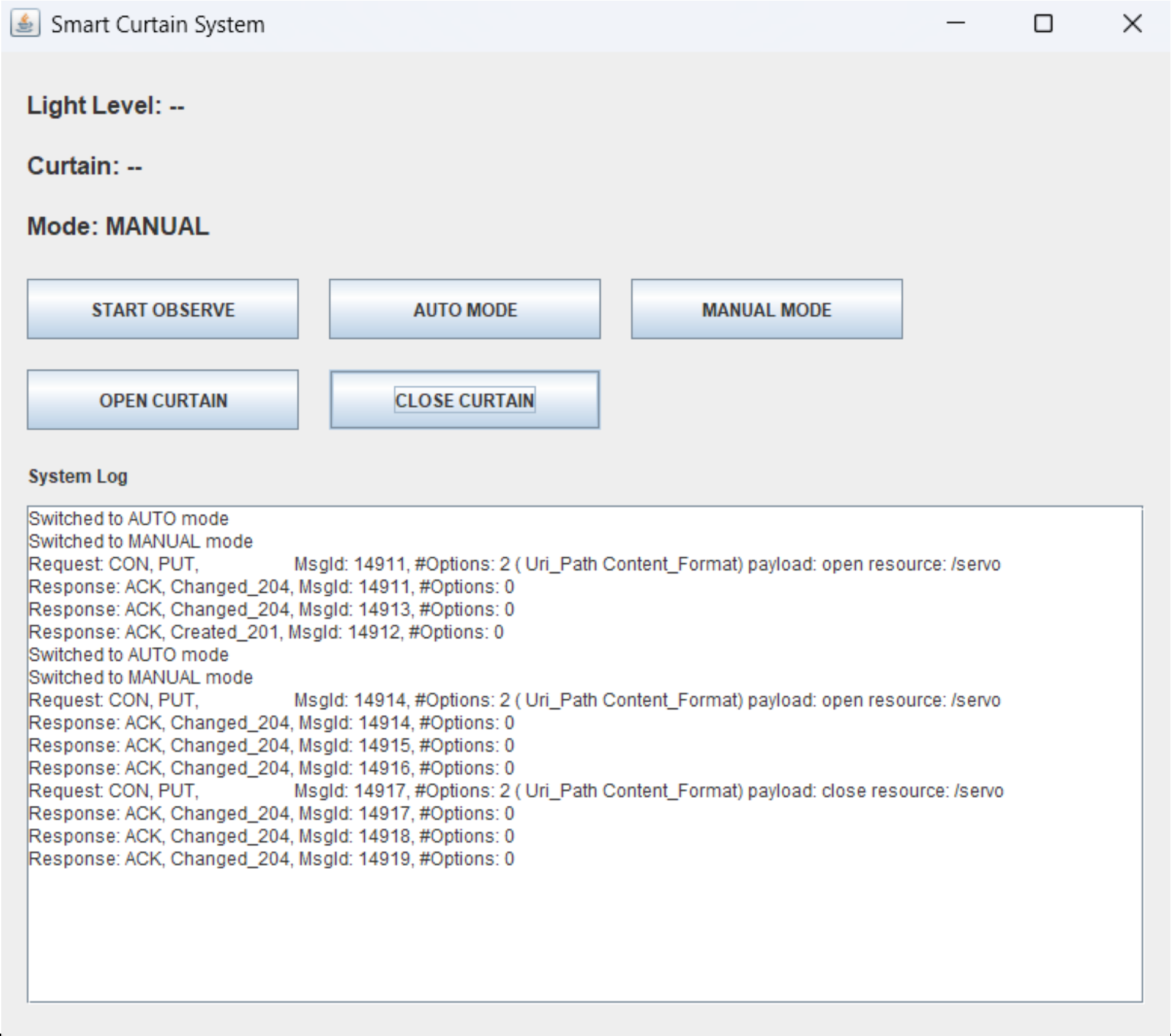
        display_text.append("AUTO: Opened curtain (bright: " + lux + " lux)\n");
    } else if (lux < 300.0) {
        // 어두움 - 커튼 닫기
        request.setPayload(new CoapData("close", CoapMediaType.text_plain));
        curtain_label.setText("Curtain: CLOSED");
        updateLCD("Light: " + String.format("%.0f", lux) + "|Curtain: CLOSE");

        display_text.append("AUTO: Closed curtain (dark: " + lux + " lux)\n");
    } else {
        return; // 중간 밝기는 변경 없음
    }

    displayRequest(request);
    clientChannel.sendMessage(request);
}
```

자동적으로 커튼 개폐를 처리

소프트웨어 구현



실행시 보이는 UI의 모습

데모 영상



레퍼런스

라즈베리파이로 자동 커튼 제어 시스템 만들기

(<https://www.aifaedu.co.kr/%EB%9D%BC%EC%A6%88%EB%B2%A0%EB%A6%AC%ED%8C%8C%EC%9D%B4%EB%A1%9C-%EC%9E%90%EB%8F%99-%EC%BB%A4%ED%8A%BC-%EC%A0%9C%EC%96%B4-%EC%8B%9C%EC%8A%A4%ED%85%9C-%EB%A7%8C%EB%93%A4%EA%B8%B0/>)

감사합니다.